Computer Science Notes

Operators and Control Flow in C Programming

Simplified notes for Secondary School Students — Form Two

Covers Student's Book pages 174 – 191

These notes explain, in very simple English, everything covered in this section of the book: logical operators, assignment operators, operator precedence, increment/decrement operators, control flow (if statements), and loops (while, for, do...while), including break and continue. Each topic from the book is explained in easy words, and an extra simple, everyday example is added after every book example, so that it is easier for a student to understand.

---

1. Logical Operators

Logical operators are used when we want to check more than one condition at the same time. They help the computer decide if something is TRUE or FALSE. In C language, we normally use them together with if statements to make decisions.

In C, the number 1 (or any number that is not zero) means TRUE, and the number 0 means FALSE.

The book uses this example: variable A holds 1 (true) and variable B holds 0 (false).

Table 4.4: List of Logical Operators

Operator	Name	Simple meaning	Example (A=1, B=0)
&&	Logical AND	TRUE only if BOTH sides are true (non-zero)	(A && B) is false
||	Logical OR	TRUE if AT LEAST ONE side is true (non-zero)	(A || B) is true
!	Logical NOT	Reverses the answer: true becomes false, false becomes true	!(A && B) is true

🧩 Simple Example — Understanding && , || , ! in daily life

&& (AND): "You will pass the exam if you READ your notes AND you SLEEP well." Both things must happen — if you miss one, you fail.
|| (OR): "You can travel to school by BUS OR by BICYCLE." Only one of them needs to be true for you to reach school.
! (NOT): If the statement is "It is raining" is TRUE, then "It is NOT raining" is FALSE. The NOT operator simply flips the answer.

Program Example 4.5 — Demonstration of logical operators

```
#include <stdio.h>
int main()
{
    int a = 5;
    int b = 20;
    int c;
    if (a && b)
```

```
    {
       printf("Line 1 - Condition is true\n");
    }
    if (a || b)
    {
        printf("Line 2 - Condition is true\n");
    }
    /* Let you, now change the value of a and b */
    a = 0;
    b = 10;
    if (a && b)
    {
        printf("Line 3 - Condition is true\n");
    }
    else
    {
        printf("Line 3 - Condition is not true\n");
    }
    if (!(a && b))
    {
        printf("Line 4 - Condition is true\n");
    }
    return 0;
}
```

Output:

```
Line 1 - Condition is true
Line 2 - Condition is true
Line 3 - Condition is not true
Line 4 - Condition is true
```

Simple explanation of the program:

- At first, a = 5 and b = 20. Both are not zero, so (a && b) is true — Line 1 is printed.
- (a || b) only needs one to be true, so Line 2 is also printed.
- Then the program changes a = 0 and b = 10. Now a is zero, so (a && b) becomes false — the else part runs and prints Line 3 as "not true".
- !(a && b) reverses that false into true, so Line 4 is printed as true.

🧩 Simple Example — A school attendance checker

```
int hasBook = 1;     // student has exercise book
int hasPen = 0;      // student has NO pen
if (hasBook && hasPen) {
   printf("Ready for class\n");
} else {
   printf("Not fully ready\n");
}
```

Since hasPen is 0 (false), (hasBook && hasPen) is false, so the program will print "Not fully ready".

How to compile and run the program (using MinGW)

The book explains that you can run the program directly using the MinGW compiler. Follow these steps:

- (i) Open Command Prompt (cmd), write the path where you saved your file, then type: gcc logical2.c -o logical2

- (ii) When you open cmd.exe, it may show a default path like C:\Users\admin>. If you saved your file on the Desktop, change the directory to Desktop using the command: cd Desktop. This uses the MinGW compiler to change your text file in Notepad into a program that can run.

```
C:\Users\admin\Desktop>cd C language

C:\Users\admin\Desktop\C language>gcc logical2.c -o logical2

C:\Users\admin\Desktop>\C language>logical2
Line 1 - Conditional is true
Line 2 - Conditional is true
Line 3 - Conditional is not true
Line 4 - Conditional is true
```

---

2. Assignment Operators

Assignment operators are used to put (store) a value inside a variable. The most common one is the equal sign (=), called simple assignment. There are also compound assignment operators, which combine a normal maths operation (like + or -) together with assignment, to save us from writing long code.

Table 4.5: List of Assignment Operators

Operator	Example	Same as	Simple meaning
=	a = b	a = b	Puts the value of b into a
+=	a += b	a = a + b	Adds b to a, and stores the answer in a
-=	a -= b	a = a - b	Subtracts b from a, and stores the answer in a
*=	a *= b	a = a * b	Multiplies a by b, and stores the answer in a
/=	a /= b	a = a / b	Divides a by b, and stores the answer in a
%=	a %= b	a = a % b	Finds the remainder of a ÷ b, and stores it in a
&=	a &= b	a = a & b	Bitwise AND, then stores result in a
^=	a ^= b	a = a ^ b	Bitwise XOR, then stores result in a
<<=	a <<= b	a = a << b	Shifts bits of a to the left by b positions

🧩 Simple Example — Everyday meaning of +=

Imagine you have 20 shillings (a = 20 pocket money). Your mother gives you 5 more shillings (b = 5).
Instead of writing: a = a + b, C allows the short-cut: a += b;
After this, a becomes 25 — exactly the same answer, just written faster.

Program Example 4.6 — Demonstration of assignment operators

```
#include <stdio.h>
int main() {
    int a = 20, b = 5;
    a += b;  // equivalent to a = a + b
    printf("a += b: %d\n", a);
    a -= b;  // equivalent to a = a - b
    printf("a -= b: %d\n", a);
    a *= b;  // equivalent to a = a * b
```

```
    printf("a *= b: %d\n", a);
    a /= b;  // equivalent to a = a / b
    printf("a /= b: %d\n", a);
    a %= b;  // equivalent to a = a % b
    printf("a %%= b: %d\n", a);
    return 0;
}
```

Output:

```
a += b:   25
a -= b:   20
a *= b:   100
a /= b:   20
a %= b:   0
```

Notice how the value of a keeps changing after every line, because each new operation is done using the answer from the line before it.

🧩 **Simple Example — Shop money example**

```
int money = 1000;   // money in your pocket
money -= 300;       // you buy bread for 300
printf("Money left: %d\n", money);   // prints 700
```

---

3. Operator Precedence in C

Operator precedence means the order in which the computer solves an expression that has many operators. Some operators are solved before others. For example, multiplication is solved before addition — just like in normal Mathematics (BODMAS).

Example from the book: in the expression x = 8 + 4 * 3;, x becomes 20, not 36. This is because multiplication (*) is done before addition (+). So 4 * 3 = 12 is calculated first, and then 8 + 12 = 20.

In the precedence table below, operators at the TOP of the table are solved FIRST, and operators at the BOTTOM are solved LAST.

Table 4.6: List of Operator Precedence

Precedence	Operator	Description	Order (Associativity)
1	() [] -> , ++ --	Postfix	Left to Right
2	+ - ! ~ ++ -- (type) * & sizeof	Unary	Right to Left
3	* / %	Multiplicative	Left to Right
4	+ -	Additive	Left to Right
5	<< >>	Shift	Left to Right
6	< <= > >=	Relational	Left to Right
7	== !=	Equality	Left to Right
8	&	Bitwise AND	Left to Right
9	^	Bitwise XOR	Left to Right

Precedence	Operator	Description	Order (Associativity)
10	|	Bitwise OR	Left to Right
11	&&	Logical AND	Left to Right
12	||	Logical OR	Left to Right
13	?:	Conditional	Right to Left
14	= += -= *= /= %= >>= <<= &= ^= |=	Assignment	Right to Left
15	,	Comma	Left to Right

Note: For Form Two, the most important rows to remember well are: Multiplicative (* / %), Additive (+ -), Relational (< > <= >=), Logical AND/OR (&& ||), and Assignment (=).

Program Example 4.7 — Demonstration of operator precedence in C

```
#include <stdio.h>
int main()
{
  int a = 40;
  int b = 20;
  int c = 10;
  int d = 5;
  int e;
  e = (a + b) * c/d;          // (40+20)*10/5 -> 60*10/5 -> 600/5 -> 120
  printf("Value of (a + b) * c / d is : %d\n", e);
  e = ((a + b)*c)/d;          // ((40+20)*10)/5 -> (60*10)/5 -> 600/5 -> 120
  printf("Value of ((a + b) * c) / d is : %d\n", e);
  e = (a + b) * (c / d);      // (40+20) * (10/5) -> 60*2 -> 120
  printf("Value of (a + b) * (c / d) is : %d\n", e);
  e = a + (b * c)/d;          // 40 + (20*10)/5 -> 40 + 200/5 -> 40 + 40 -> 80
  printf("Value of a + (b * c) / d is : %d\n", e);
  return 0;
}
```

Output:

```
value of (a + b) * c / d is : 120
value of ((a + b) * c) / d is : 120
value of (a + b) * (c / d) is : 120
value of a + (b * c) / d is : 80
```

🧩 Simple Example — Simple maths order example

```
int result = 2 + 3 * 4;
printf("%d", result);   // Multiplication happens first: 3*4=12, then 2+12=14
```

The computer never just reads left to right — it follows the precedence rules, exactly like BODMAS in Mathematics.

---

4. Increment (++) and Decrement (--) Operators

The increment (++) and decrement (--) operators increase or decrease the value of a variable by 1.

Increment operator (++)

The increment operator adds 1 to the value of a variable.

- Prefix (++a): adds 1 FIRST, then uses the new value.
- Postfix (a++): uses the CURRENT value first, and adds 1 AFTER.

(a) Prefix increment (++a)

```
int a = 15;
int result = ++a;   // a becomes 16 first, then result = 16
printf("%d ", result);
```

Output:
16

(b) Postfix increment (a++)

```
int a = 15;
int result = a++;   // result gets 15 first, then a becomes 16
printf("%d ", result);
```

Output:
15

Decrement operator (--)

The decrement operator subtracts 1 from the value of a variable.

- Prefix (--a): subtracts 1 FIRST, then uses the new value.
- Postfix (a--): uses the CURRENT value first, then subtracts 1 AFTER.

(a) Prefix decrement (--a)

```
int a = 15;
int result = --a;   // a becomes 14 first, then result = 14
printf("%d ", result);
```

Output:
14

(b) Postfix decrement (a--)

```
int a = 15;
int result = a--;   // result gets 15 first, then a becomes 14
printf("%d ", result);
```

Output:
15

🧩 Simple Example — Counting football goals

int goals = 0;

goals++; // a goal is scored, goals becomes 1

goals++; // another goal, goals becomes 2

printf("Total goals: %d", goals); // prints 2

Every time ++ is used, it is like adding one more goal to the scoreboard.

Exercise 4.4 (Practice Questions)

Below are the practice questions from the book. Students should try to solve them using what has been learnt about logical, assignment, precedence, and increment/decrement operators.

- 1. What will be the output of this C code? (uses int a = 5, b = 3; int result = a + b * 2;) — Hint: remember multiplication is done before addition.
- 2. Write a C program that inputs two numbers and prints their sum, difference, product, and quotient using the appropriate arithmetic operators.
- 3. What will be the output of this code? (uses int num = 5; printf with num++ then ++num) — Hint: remember the difference between prefix and postfix.
- 4. Write a C program that checks if a given number is greater than 10 and less than 20. Print "In range" if true, otherwise print "Out of range".
- 5. What will be the output of this code? (uses int a = 10, b = 5, result; result = a %% b + a / b;) — Hint: work out the % (remainder) and / (division) parts first, then add them.
- 6. If a = 100 and the statement b = a--; is executed, what will be the values of a and b after execution? — Hint: postfix decrement gives the old value to b first.

---

5. Control Flow in C

Control flow means the order in which the instructions (statements) of a program are carried out (executed). Control flow is very important because it allows a program to make decisions, repeat actions, and react differently depending on the situation — almost like how we make decisions in daily life.

Introduction to Control Flow in C

There are three main types of control flow:

(a) Sequential flow

This is the normal, default way a program runs — each statement is carried out in order, from top to bottom, one after another.

```
int a = 10;
int b = 20;
int sum = a + b;
printf("Sum is: %d\n", sum);
```

In this example, the computer first stores 10 in a, then 20 in b, then adds them together, and finally prints the answer — always in that order.

(b) Selection flow

This type of control flow makes decisions based on conditions. It allows the program to run different blocks of code depending on whether a condition is true or false.

(c) Iterative flow

This type of control flow repeats a block of code many times. It is commonly used in loops such as for, while, and do-while.

🧩 Simple Example — Comparing the three flows to daily life

Sequential: Waking up, then brushing teeth, then eating breakfast, then going to school — always in that order.
Selection: "If it is raining, carry an umbrella. Otherwise, do not." — a decision is made based on a condition.
Iterative: "Keep sweeping the classroom until it is clean." — the same action (sweeping) is repeated many times.

---

6. Selection Flow — Making Decisions with if Statements

Selection flow uses conditional statements to make decisions in a program. This is important because it allows a program to behave differently depending on the situation. Using conditional statements, a program can:

- (i) Respond to user input,
- (ii) Check if certain conditions are met,
- (iii) Execute code based on various scenarios.

Example: a program can ask for a user's age and tell them whether they are eligible to vote, based on the age they typed in.

(a) if Statement

The if statement checks a condition. If the condition is true, the code block inside the curly brackets { } runs. If it is false, that code is simply skipped.

Syntax:

```
if (condition) {
  // Code to execute if the condition is true
}
```

Example:

```
#include <stdio.h>
int main() {
    int num = 10;
    if (num > 0) {
        printf("Positive number\n");
    }
    return 0;
}
```

Explanation: This program checks if num is greater than 0. Since 10 is greater than 0, the condition is true, so it prints "Positive number".

> **Task:** Task: Write a program that checks whether a number the user enters is positive, negative, or zero.
> Hint: Use if to check if the number is greater than zero, and use else to check for negative numbers.

🧩 Simple Example — Checking if it is time for a break

int minutesLeft = 0;
if (minutesLeft == 0) {

```
  printf("Time for a break!\n");
}
```
Since minutesLeft is 0, the condition is true, so the message will be printed.

(b) if-else Statement

The if-else statement lets a program choose between two paths: one path runs when the condition is true, and a different path runs when the condition is false.

Syntax:

```
if (condition) {
      // Code to execute if the condition is true
} else {
      // Code to execute if the condition is false
}
```

Example:

```
#include <stdio.h>
int main() {
    int num = -5;
    if (num >= 0) {
        printf("Positive or zero\n");
    } else {
        printf("Negative number\n");
    }
    return 0;
}
```

Explanation: This program checks if num is greater than or equal to 0. Since num is -5, the condition is false, so the else part runs, and "Negative number" is printed.

> **Task:** Task: Write a program that asks the user for their age and prints whether they are eligible to vote. (Assume the voting age is 18). Hint: Use an if-else statement to check if the age is greater than or equal to 18.

🧩 Simple Example — Checking exam results

```
int marks = 45;
if (marks >= 50) {
  printf("You passed\n");
} else {
  printf("You failed\n");
}
```
Since 45 is less than 50, the program will print "You failed".

(c) else-if Ladder

The else-if ladder is used to check many conditions, one after another. It is useful when there are several possible outcomes, depending on different conditions.

Syntax:

```
if (condition1) {
    // Code for condition1
} else if (condition2) {
```

```
    // Code for condition2
} else {
    // Code if no conditions are true
}
```

Example:

```
#include <stdio.h>
int main() {
    int num = 25;
    if (num > 30) {
        printf("Greater than 30\n");
    } else if (num > 20) {
        printf("Greater than 20 but less than or equal to 30\n");
    } else {
        printf("20 or less\n");
    }
    return 0;
}
```

Explanation: This program checks if num is greater than 30. Since it is not, it then checks if num is greater than 20. Since 25 is greater than 20, it prints "Greater than 20 but less than or equal to 30". If both conditions had been false, the last else block would have run instead.

> **Task:** Task: Write a program that takes a score from the user and prints the corresponding grade: (a) 90–100: A (b) 80–89: B (c) 70–79: C (d) 60–69: D (e) Below 60: F. Hint: Use an else-if ladder to check the score ranges.

🧩 **Simple Example — Bus fare depending on age**

```
int age = 12;
if (age < 5) {
   printf("Free ride\n");
} else if (age <= 17) {
   printf("Half fare\n");
} else {
   printf("Full fare\n");
}
```

Since age is 12, which is not less than 5 but is 17 or below, the program prints "Half fare".

Exercise 4.5 (Practice Questions)

- 1. The following program checks whether a number is positive or negative, but there is an error (uses int number = -5; if (number > 0) ... else ...). Identify and fix the issue. Hint: look carefully at the curly brackets { } around the else block.
- 2. Write a C program that checks if a given number is even or odd using an if-else statement.
- 3. Write a C program that reads a student's marks and prints their grade using if-else statements: A for marks >= 85, B for marks >= 70, C for marks >= 50, D for marks < 50.
- 4. Write a C program that checks if a number entered by a user is between 10 and 20 (inclusive) using if-else statements. If it is, print "In range"; otherwise, print "Out of range".

- 5. Write a C program that checks if a number is greater than zero and less than 100. If it is, print "Valid number"; otherwise, print "Invalid number".

---

7. Loops

A loop is a statement that allows a group of instructions to be repeated many times, instead of writing the same code again and again.

Figure 4.2 in the book shows the general flowchart for a loop: the program checks a Condition. If the condition is true, it runs the Conditional Code, and then goes back to check the condition again. This repeats again and again until the condition becomes false — then the loop stops.

C offers three types of loops:

- (i) while loop: Repeats a block of code as long as a condition remains true. The condition is checked BEFORE the loop body runs.
- (ii) for loop: Repeats a block of code a specific number of times, and keeps the counting (loop control) organised in one line.
- (iii) do...while loop: Works like the while loop, but checks the condition AFTER running the loop body — so the code always runs at least once.

🧩 Simple Example — Loops in daily life

while loop: "While there is still water in the bucket, keep pouring it into cups."
for loop: "Do 10 push-ups." — you know exactly how many times to repeat before you start.
do-while loop: "Taste the soup, and if it needs more salt, add salt and taste again." — you always taste at least once, before deciding whether to repeat.

(a) while Loop

In C, a while loop keeps running a statement or a block of statements for as long as the given condition stays true.

Syntax:

```
while (condition) {
    statement(s);
}
```

The condition can be any expression, and it is considered true if it works out to a value that is not zero. The statement(s) can be a single line or many lines inside { }. The loop keeps running while the condition is true. Once the condition becomes false, the program moves to the first line after the loop.

Figure 4.3 in the book shows this flow: the Condition is checked; if true, the code block runs and then goes back to check the Condition again; if false, the loop stops.

> **Note:** An important feature of the while loop is that it may not run at all. If the condition is false right from the start, the loop body is skipped completely, and the program continues with the line after the loop.

Program Example 4.8 — Demonstration of while loop execution in C

```
#include <stdio.h>
int main() {
    /* local variable definition */
    int a = 10;
    /* while loop execution */
    while (a < 20) {
        printf("value of a: %d\n", a);
        a++;
    }
    return 0;
}
```

Output:

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

Explanation: The loop keeps checking if a is less than 20. As long as this is true, it prints the value of a, and then increases a by 1 (a++). Once a becomes 20, the condition (a < 20) becomes false, and the loop stops.

🧩 **Simple Example — Counting down while loop**

```
int count = 5;
while (count > 0) {
   printf("%d\n", count);
   count--;
}
```

This will print 5, 4, 3, 2, 1 — and then stop, because once count becomes 0, the condition (count > 0) is false.

(b) for Loop

A for loop is a control structure used to repeat a block of code a specific number of times, in a simple and organised way.

Syntax:

```
for (initialisation; condition; increment)
{
    statement(s);
}
```

The for loop works in three steps:

- (i) Initialisation: This runs once, before the loop starts. It is used to set up the loop control variable. This part is optional, but must be followed by a semicolon.
- (ii) Condition check: Before every repetition (iteration), the condition is checked. If it is true, the loop body runs. If it is false, the loop stops, and the program moves to the next line after the loop.

- (iii) Increment/update: After the loop body runs, the control moves to the increment step, which updates the loop control variable. The condition is checked again, and the process repeats.

Figure 4.4 in the book shows this flow: Init runs once → Condition is checked → if true, the code block runs, then Increment happens, then the Condition is checked again → this repeats until the Condition becomes false, and then the loop stops.

Program Example 4.9 — Demonstration of for-loop execution in C

```
#include <stdio.h>
int main() {
    /* for loop execution */
    for (int a = 10; a < 20; a = a + 1) {
        // Print the value of 'a' in each iteration
        printf("value of a: %d\n", a);
    }
    return 0;  // Indicating successful program termination
}
```

Output:

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

🧩 **Simple Example — Printing your name 3 times**

```
for (int i = 1; i <= 3; i++) {
   printf("Vincent\n");
}
```

This will print the word "Vincent" three times, because i starts at 1, and the loop keeps running while i is less than or equal to 3, adding 1 to i each time.

(c) do...while Loop

Unlike the while and for loops, which check the condition at the beginning, the do...while loop in C checks its condition at the END of the loop.

Syntax:

```
do
{
    statement(s);
} while (condition);
```

> **Note:** The condition is checked at the end of the loop, which means the statements inside the loop will run at least once before the condition is even checked. If the condition is true, control goes back to the start of the loop and the statements run again. This continues until the condition becomes false.

Figure 4.5 in the book shows this flow: the code block runs FIRST, then the Condition is checked; if true, it goes back and the code block runs again; if false, the loop stops.

Program Example 4.10 — Demonstration of do...while loop execution in C

```
#include <stdio.h> // Include the standard input-output library
int main()
{
  // Local variable definition
  int a = 10;  // Initialise the variable 'a' to 10
  // do...while loop execution
  do
  {
    // Print the current value of 'a'
    printf("value of a: %d\n", a);
    // Increment 'a' by 1
    a = a + 1;
  } while (a < 20); // Continue looping as long as 'a' is less than 20
  return 0;
}
```

Output:

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

🧩 **Simple Example — Asking a student to enter a positive number**

```
int number;
do {
   printf("Enter a positive number: ");
   scanf("%d", &number);
} while (number <= 0);
```

This keeps asking the student to enter a number, and will always ask at least once, even before checking whether the number is positive.

Exercise 4.6 (Practice Questions)

- 1. The program below is intended to print numbers from 1 to 5, but there is an error in the loop condition (for (i = 1; i < 5; i++)). Identify and fix the errors. Hint: think about whether it should be < 5 or <= 5 to include the number 5.
- 2. Write a C program using a for loop that prints all even numbers from 1 to 20.
- 3. Write a C program using a while loop that calculates and prints the sum of numbers from 1 to 10.
- 4. Write a C program that uses a for loop to print the following pattern:

```
*
**
***
****
*****
```

- 5. Write a C program that uses a do...while loop to print the numbers from 1 to 10.

---

8. Using break and continue in Loops

In C programming, break and continue are special statements used inside loops to change how the loop runs.

(i) break

The break statement completely stops (terminates) the loop, and exits it immediately — no matter what the condition says.

Example:

```
#include <stdio.h>
int main() {
    for (int i = 0; i < 5; i++) {
        if (i == 3) {
            break;   // stop the loop when i equals 3
        }
        printf("%d ", i);
    }
    return 0;
}
```

Output:

```
0 1 2
```

Explanation: The loop starts printing i (0, 1, 2). As soon as i becomes 3, the break statement runs, and the loop stops completely, even though i has not yet reached 5.

🧩 Simple Example — Searching for a friend in a queue

```
for (int position = 1; position <= 10; position++) {
  if (position == 4) {
     printf("Found my friend at position 4\n");
     break;   // stop searching once found
  }
}
```

Once the friend is found at position 4, there is no need to keep checking positions 5 to 10 — break stops the loop right away.

(ii) continue

The continue statement skips only the CURRENT repetition (iteration), and moves the loop straight to the next one — the loop itself keeps running.

Example:

```
#include <stdio.h>
int main() {
    for (int i = 0; i < 5; i++) {
        if (i == 3) {
            continue;   // skip printing when i equals 3
```

```
        }
        printf("%d ", i);
    }
    return 0;
}
```

Output:
```
0 1 2 4
```

Explanation: The loop prints i for 0, 1, 2. When i becomes 3, continue is triggered, so printf is skipped just for that one round — but the loop does not stop. It moves on and prints 4 when i becomes 4.

🧩 Simple Example — Skipping a rotten fruit while counting

```
for (int fruit = 1; fruit <= 5; fruit++) {
   if (fruit == 3) {
      continue;   // skip the rotten fruit, but keep counting the rest
   }
   printf("Counting good fruit number %d\n", fruit);
}
```

Fruit number 3 is skipped (maybe it is rotten), but the loop still continues and counts fruits 1, 2, 4, and 5.

Key difference to remember: break stops the WHOLE loop immediately, while continue only skips the CURRENT round and lets the loop carry on.

End-of-Topic Summary

- Logical operators (&&, ||, !) combine or reverse conditions.
- Assignment operators (=, +=, -=, *=, /=, %=, etc.) store and update values in variables.
- Operator precedence decides which part of an expression is calculated first, just like BODMAS.
- Increment (++) and decrement (--) operators add or subtract 1, and can be used as prefix or postfix.
- Control flow can be sequential (in order), selective (if / if-else / else-if), or iterative (loops).
- Loops (while, for, do...while) repeat a block of code, and break/continue can change how a loop behaves.

Students are encouraged to type and run every example program themselves using a C compiler (such as MinGW/gcc), to see the output with their own eyes and understand each concept better.